Q1. Bubble Sort

```
#include <iostream>
using namespace std;
int main() {
    int arr[] = {5,4,3,2,1};
    int n = 5;
    for(int i=0; i<n-1; i++) {
        for(int j=0; j<n-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
    cout << "Sorted IDs: ";
    for(int i=0; i<n; i++) cout << arr[i] << " ";
    return 0;
}
```

Q2. Insertion Sort

```
#include <iostream>
using namespace std;
int main() {
    int arr[] = {5,4,3,2,1};
    int n = 5;
    for(int i=1; i<n; i++) {
        int key = arr[i];
        int j = i - 1;
        while(j >= 0 && arr[j] > key) {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
    cout << "Sorted IDs: ";
    for(int i=0; i<n; i++) cout << arr[i] << " ";
    return 0;
}
```

Q3. Selection Sort

```
#include <iostream>
using namespace std;
int main() {
    int arr[] = {5,4,3,2,1};
    int n = 5;
    for(int i=0; i<n-1; i++) {
        int minIndex = i;
        for(int j=i+1; j<n; j++) {
            if(arr[j] < arr[minIndex])
```

```
            minIndex = j;
        }
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
    cout << "Sorted IDs: ";
    for(int i=0; i<n; i++) cout << arr[i] << " ";
    return 0;
}
```

Q4. Linked List

```
#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* next;
};
int main() {
    Node* head = new Node{111, nullptr};
    head->next = new Node{123, nullptr};
    head->next->next = new Node{124, nullptr};
    Node* temp = head;
    cout << "Linked List: ";
    while(temp != nullptr) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL";
    return 0;
}
```

Q5. Graph (Adjacency List & Matrix)

```
#include <iostream>
using namespace std;
int main() {
    int n = 6;
    int adj[7][7] = {0};
    adj[1][2] = adj[2][1] = 1;
    adj[1][5] = adj[5][1] = 1;
    adj[1][6] = adj[6][1] = 1;
    adj[2][3] = adj[3][2] = 1;
    adj[2][5] = adj[5][2] = 1;
    adj[3][4] = adj[4][3] = 1;
    adj[3][5] = adj[5][3] = 1;
    adj[4][5] = adj[5][4] = 1;
    adj[5][6] = adj[6][5] = 1;
    cout << "Adjacency Matrix:\n";
    for(int i=1; i<=n; i++) {
        for(int j=1; j<=n; j++)
            cout << adj[i][j] << " ";
        cout << endl;
```

```
    }
    cout << "\nAdjacency List:\n";
    for(int i=1; i<=n; i++) {
        cout << i << " -> ";
        for(int j=1; j<=n; j++)
            if(adj[i][j]==1) cout << j << " ";
        cout << endl;
    }
    return 0;
}
```

Q6. Binary Tree

```
#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* left;
    Node* right;
};
Node* newNode(int data) {
    Node* node = new Node;
    node->data = data;
    node->left = node->right = nullptr;
    return node;
}
void inorder(Node* root) {
    if(root == nullptr) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}
int main() {
    Node* root = newNode(50);
    root->left = newNode(30);
    root->right = newNode(70);
    root->left->left = newNode(20);
    root->left->right = newNode(40);
    root->right->left = newNode(60);
    cout << "Inorder Traversal: ";
    inorder(root);
    return 0;
}
```

Q7. Hashing

```
#include <iostream>
using namespace std;
int main() {
    int tableSize = 3;
    int hashTable[3];
    for(int i=0; i<tableSize; i++) hashTable[i] = -1;
    int keys[] = {1,2,3,4};
    for(int i=0; i<4; i++) {
```

```
        int index = keys[i] % tableSize;
        if(hashTable[index] == -1)
            hashTable[index] = keys[i];
        else
            cout << "Collision at index " << index << " for key " << keys[i] << endl;
    }
    cout << "\nFinal Hash Table:\n";
    for(int i=0; i<tableSize; i++)
        cout << i << " : " << hashTable[i] << endl;
    return 0;
}
```

Q8. Graph (Traffic System Matrix)

```
#include <iostream>
using namespace std;
int main() {
    int n = 3;
    int adj[3][3] = {
        {0,1,1},
        {1,0,1},
        {1,1,0}
    };
    cout << "Adjacency Matrix:\n";
    for(int i=0; i<n; i++) {
        for(int j=0; j<n; j++)
            cout << adj[i][j] << " ";
        cout << endl;
    }
    return 0;
}
```